

PROJECT MANUAL

ShadiMatch User and Developer Manual

Complete guide for running, testing, and maintaining the ShadiMatch matrimony web application.

Frontend: React + Vite

Backend: Node.js + Express + Sequelize + MySQL

Realtime Chat: Socket.IO

Document: Generated for local project handover and testing.

Table of Contents

1. Project Overview
2. Main Features
3. System Requirements
4. Setup and Run
5. Database and Tables
6. Registration and OTP Login
7. Test Users
8. Matching Rules
9. Realtime Chat and Offline Email
10. Frontend Pages
11. API Reference
12. Troubleshooting
13. Production Checklist

1. Project Overview

ShadiMatch is a matrimony-style matching application. It allows users to register, verify login using mobile OTP, browse compatible profiles, like or pass profiles, start conversations, and receive offline message email notifications.

The application is split into two parts:

- **Frontend:** React app inside the `src` folder.
- **Backend:** Express API and Socket.IO server inside the `server` folder.

Folder Structure

Path	Purpose
<code>src/pages</code>	React pages such as Home, Register, Login, Browse, Messages, Profile, Likes, and footer information pages.
<code>src/components</code>	Reusable UI components such as Navbar and ProfileCard.
<code>src/services/api.js</code>	Axios API client and Socket.IO client setup.
<code>server/routes</code>	Express routes for auth, profiles, matches, and messages.
<code>server/models</code>	Sequelize models and relationships.
<code>server/socket</code>	Socket.IO connection and room handling.
<code>server/services</code>	Presence and email notification services.

2. Main Features

- Mobile number collected during registration.
- OTP based login using registered mobile number.
- JWT authentication for protected API routes.
- 10 male and 10 female seeded test users.
- Male test user sees 10 female matches.
- Female test user sees 10 male matches.
- Like, pass, views, and conversation start support.

- Realtime chat using Socket.IO rooms.
- Email notification when recipient is offline.
- Interactive footer pages for About, Blog, Careers, Help, Contact, FAQ, Privacy, Terms, and Safety.

3. System Requirements

Tool	Recommended	Notes
Node.js	20 or newer	Used for React build and Express API.
MySQL	8 or compatible	Database name should be <code>shadiDB</code> .
npm	Bundled with Node	Used to install dependencies and run scripts.
Browser	Chrome or Edge	Used to test the frontend.

4. Setup and Run

Install Dependencies

```
npm install
```

Configure Environment

Edit `.env` in the project root:

```
PORT=5000
CLIENT_URL=http://127.0.0.1:5173,http://localhost:5173

DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=shadiDB
DB_USER=root
DB_PASSWORD=

JWT_SECRET=change-this-secret-before-production
```

Start Backend

```
npm run server
```

Backend URL:

```
http://127.0.0.1:5000
```

Start Frontend

```
npm run dev
```

Frontend URL:

```
http://127.0.0.1:5173
```

Build Frontend

```
npm run build
```

5. Database and Tables

The backend uses Sequelize ORM. On startup, the server runs:

```
sequelize.sync({ alter: true })
```

This creates or updates the required MySQL tables automatically.

Main Tables

Table	Purpose
Users	Stores account, mobile, OTP, gender, lookingFor, DOB, and auth data.
Profiles	Stores religion, caste, location, education, occupation, height, bio, and interests.
Photos	Stores profile photo URLs.
Likes	Stores like/pass actions.
ProfileViews	Stores profile view records.
Conversations	Stores chat conversation records.
ConversationMembers	Maps users to conversations.
Messages	Stores chat messages.

6. Registration and OTP Login

Registration Flow

1. User opens `/register`.
2. User enters full name, email, mobile number, password, DOB, religion, and preferences.
3. Backend creates the user and profile.
4. User is redirected to `/login`.

OTP Login Flow

1. User opens `/login`.
2. User enters registered mobile number.
3. Backend generates a 6 digit OTP.
4. User enters OTP.
5. Backend verifies OTP and returns JWT token.
6. Frontend stores token in `localStorage` and redirects to `/browse`.

Development note: OTP is currently returned as `devOtp` and shown in the UI. For production, connect an SMS provider and remove OTP display from the frontend.

7. Test Users

All seeded users use password `password123` for legacy API login testing. For UI login, use mobile OTP.

Female Test Users

Name	Email	Mobile
Priya Sharma	priya@example.com	9000000001
Anjali Verma	anjali@example.com	9000000002
Neha Gupta	neha@example.com	9000000003
Megha Singh	megha@example.com	9000000007
Kavya Iyer	kavya@example.com	9000000008
Isha Kapoor	isha@example.com	9000000009
Ritika Jain	ritika@example.com	9000000010
Sneha Nair	sneha@example.com	9000000011
Aditi Rao	aditi@example.com	9000000012
Pooja Mehta	pooja@example.com	9000000013

Male Test Users

Name	Email	Mobile
Raj Patel	raj@example.com	9000000004
Arjun Kumar	arjun@example.com	9000000005
Vikram Singh	vikram@example.com	9000000006
Aditya Reddy	aditya@example.com	9000000014
Rohan Mishra	rohan@example.com	9000000015
Nikhil Joshi	nikhil@example.com	9000000016
Karan Malhotra	karan@example.com	9000000017
Sameer Khan	sameer@example.com	9000000018
Manav Shah	manav@example.com	9000000019
Dev Sharma	dev@example.com	9000000020

8. Matching Rules

The match API returns compatible profiles based on gender preference:

- If logged in user is male and looking for female, API returns female profiles looking for male.
- If logged in user is female and looking for male, API returns male profiles looking for female.
- By default, the API returns 10 matches for testing.
- Filters can apply age, location, and religion.

Expected Test Result

Login User	Expected Matches
Raj Patel	10 female profiles
Priya Sharma	10 male profiles

9. Realtime Chat and Offline Email

Realtime Chat

1. User opens a matched profile and clicks message.
2. Frontend calls `POST /api/messages/start/:profileId`.
3. Backend creates or reuses a conversation.
4. Messages page joins the Socket.IO conversation room.
5. New messages are broadcast with `message:new`.

Offline Email Notification

If the recipient is not connected to Socket.IO, the backend sends an email notification with the message content.

```
SMTP_HOST=smtp.gmail.com  
SMTP_PORT=587  
SMTP_SECURE=false  
SMTP_USER=your_email@gmail.com  
SMTP_PASS=your_app_password  
EMAIL_FROM=your_email@gmail.com
```

If SMTP settings are empty, the backend logs the email notification in the server console instead of sending it.

10. Frontend Pages

Route	Page	Description
/	Home	Landing page with footer links.
/register	Register	Create profile and mobile number.
/login	Login	Mobile OTP login.
/browse	Browse	Shows matched profiles.
/likes	Likes	Likes and views list.
/messages	Messages	Realtime chat UI.
/profile	Profile	Edit logged in user profile.
/info/:slug	Footer Page	Interactive content pages for footer menu.

11. API Reference

Auth APIs

Method	Path	Purpose
POST	/api/auth/register	Create account and profile.
POST	/api/auth/send-otp	Generate OTP for registered mobile.
POST	/api/auth/verify-otp	Verify OTP and return JWT.
POST	/api/auth/login	Legacy email/password login for API testing.
GET	/api/auth/me	Return logged in user and profile.

Match APIs

Method	Path	Purpose
GET	<code>/api/matches</code>	List compatible matches.
POST	<code>/api/matches/:profileId/like</code>	Like a profile.
POST	<code>/api/matches/:profileId/pass</code>	Pass a profile.
GET	<code>/api/matches/likes</code>	List received likes.
GET	<code>/api/matches/views</code>	List profile viewers.

Message APIs

Method	Path	Purpose
POST	<code>/api/messages/start/:profileId</code>	Create or open conversation.
GET	<code>/api/messages/conversations</code>	List conversations.
GET	<code>/api/messages/conversations/:conversationId</code>	List messages.
POST	<code>/api/messages/conversations/:conversationId</code>	Send message.

12. Troubleshooting

CORS Error

- Ensure backend is running on `http://127.0.0.1:5000`.
- Ensure frontend origin is listed in `CLIENT_URL`.
- Restart backend after changing `.env`.

Database Connection Fails

- Confirm MySQL is running.
- Confirm database `shadiDB` exists.
- Check `DB_USER` and `DB_PASSWORD` in `.env`.

OTP Not Working

- Use a registered mobile number.
- Check Development OTP shown on login page.
- OTP expires after 5 minutes.

No Matches Showing

- Restart backend so seed users are inserted.
- Log in with one of the seeded test mobile numbers.
- Clear filters in Browse page.

13. Production Checklist

- Change `JWT_SECRET` to a strong private value.
- Disable returning `devOtp` from `/api/auth/send-otp`.
- Connect a real SMS provider for OTP delivery.
- Configure SMTP credentials for offline email notification.
- Use HTTPS for frontend and backend.
- Move secrets out of committed files.
- Add rate limiting for OTP and login APIs.

- Add stronger validation for profile data and uploaded images.
- Replace demo image URLs and seed data with production content.
- Use migrations instead of `sequelize.sync({ alter: true })` for production database changes.

End of manual.